

Appunti — Gestione File: find, tar, rsync, Backup

Modulo: 2C | **Corso:** Lab Amministrazione di Sistemi T

find

find esplora il filesystem in tempo reale cercando file che rispettano dei criteri.

find <dove> <criteri> <azione>

Criteri

Criterio	Esempio	Significato
-name	-name '*.log'	Nome del file (wildcard tra apici)
-type	-type f / -type d	Tipo: f=file, d=directory, l=link
-size	-size +100k	Dimensione: + maggiore di, - minore di
-mtime	-mtime -2	Modificato negli ultimi 2 giorni (-=entro, +=più di)
-user	-user alice	Proprietario specifico
-nouser	-nouser	Nessun utente corrispondente (file "orfani")
-perm	-perm -2000	Permessi specifici attivi (es. SGID)

Azioni

Azione	Significato
-print	Stampa il path del file trovato (default se non specifichi nulla)
-exec comando {} \;	Esegue un comando su ogni file trovato
-delete	Cancella ogni file trovato
-ls	Stampa dettagli stile ls -l per ogni file trovato

-exec — eseguire comandi sui risultati

```
find /usr/src -name '*.c' -size +100k -exec cat {} \;  
#                                     ^^  ^^  
#                                     |    \; = fine del comando  
#                                     {} = nome del file trovato
```

Per ogni file trovato, {} viene sostituito con il path del file e il comando viene eseguito. Il \; è obbligatorio per indicare a find dove finisce il comando (il backslash fa l'escape del ; altrimenti interpretato dalla shell).

locate

Perché non funziona nella VM

locate non è installato di default su Debian. Installalo con:

```
sudo apt install plocate
```

Dopo l'installazione, aggiorna il database prima del primo uso:

```
sudo updatedb  
locate passwd
```

file — identificazione del contenuto

In Linux le estensioni (.txt, .jpg...) sono puramente decorative per l'utente — il sistema non le usa per decidere cosa è un file. Il comando file identifica il vero contenuto.

```
file /bin/bash  
# → /bin/bash: ELF 64-bit LSB pie executable, x86-64, ...
```

```
file /etc/passwd  
# → /etc/passwd: ASCII text
```

```
file foto.jpg.txt # anche se il nome dice .txt  
# → foto.jpg.txt: JPEG image data
```

Cosa estraggo dall'output? Il tipo concreto del file, indipendentemente dal nome. È utile quando ricevi un file senza estensione, o con estensione sbagliata, o quando vuoi sapere se un binario è davvero eseguibile.

I tre livelli che usa internamente (non devi memorizzarli, ma capire la logica): 1. Controlla se il file è vuoto o è un file speciale (tipo /dev/zero) 2. Legge

i primi byte del file e li confronta con un database di **magic number** — sequenze di byte caratteristiche di ogni formato. Un file JPEG inizia sempre con FF D8 FF, un eseguibile Linux con 7F 45 4C 46. Con questi “impronte” identifica il formato. 3. Se nessun magic number corrisponde, prova a capire empiricamente se è testo e in che lingua o linguaggio di programmazione

dd — trasferimento dati a basso livello

Cos'è /dev/zero e cosa stai copiando

/dev/zero è un **file speciale virtuale** — non esiste su disco, è fornito direttamente dal kernel. Quando lo leggi, produce un flusso infinito di byte zero (0x00). Non c'è un “contenuto” pre-esistente: il kernel li genera al momento.

Quindi nell'esercizio:

```
dd if=/dev/zero of=file.out bs=4k count=16k
```

Stai leggendo byte zero dal kernel (sorgente infinita) e scrivendoli in file.out. Il risultato è un file da 64MB riempito di zeri — spazio su disco riservato, ma senza dati significativi.

Perché serve? Per riservare spazio in anticipo. Se sai che un'operazione avrà bisogno di 64MB, li puoi prenotare ora e garantire che lo spazio ci sia quando serve.

File aperti — fuser e lsof

fuser non funziona nella VM

Installalo con:

```
sudo apt install psmisc
```

I due script — cosa fanno e cosa cambia

Prima di capire la differenza, vediamo la parte che confonde: `count=$(( $(echo $RANDOM | rev | cut -c1) + 1 ))`.

Smontata pezzo per pezzo: - `$RANDOM` → numero casuale tra 0 e 32767, es. 19847 - `echo 19847 | rev` → inverte la stringa: 74891 - `cut -c1` → prende solo il primo carattere: 7 - `=$(( 7 + 1 ))` → somma 1: 8

Risultato: un numero casuale tra 1 e 10. Ogni ciclo scrive una quantità casuale di KB.

Perché non c'è of=? Perché il redirect >> sostituisce of=. dd senza of= scrive su stdout, e >> output redirige stdout nel file output. È la stessa cosa di of=output, solo scritta diversamente.

La differenza tra v1 e v2:

v1 – >> output è DENTRO il loop

while sleep 1; **do**

dd if=/dev/zero bs=1k count=... >> output *# apre file, scrive, chiude file*
done

v2 – >> output è FUORI dal loop

while sleep 1; **do**

dd if=/dev/zero bs=1k count=... *# scrive su stdout*
done >> output *# stdout → file, aperto una sola volta*

In v1: ogni iterazione apre output, scrive, chiude output. In v2: output viene aperto una volta all'avvio e rimane aperto per tutto il ciclo.

Conseguenza pratica: se mentre v2 è in esecuzione rinomini o cancelli il file output, lo script continua a scrivere sul vecchio inode (il file esiste ancora perché è aperto da un processo). Con v1, al ciclo successivo prova a riaprire il file per nome — se non c'è, ne crea uno nuovo.

fuser e lsof in pratica

fuser output *# stampa i PID dei processi che hanno output aperto*
lsof output *# mostra dettagli: processo, PID, utente, descrittore, pa*
lsof -p 1234 *# tutti i file aperti dal processo con PID 1234*

Uso tipico: “perché non riesco a smontare questo disco?” → lsof /mnt/disco
→ scopri quale processo ha ancora file aperti lì.

tar — archiviazione

Le lettere — come si legge cvpf

Le opzioni di tar sono singole lettere che si concatenano senza spazio (il trattino è facoltativo):

Lettera	Significato
c	C rea archivio
x	E xtrai da archivio
t	Elenca contenuto (t able of contents)
r	Aggiunge file a archivio esistente

Lettera	Significato
v	V erboso — stampa ogni file processato
p	P reserva permessi e ownership
f	F ile — specifica il nome dell'archivio (deve essere l'ultima, perché l'argomento la segue)
z	Compressione g zip
j	Compressione b zip2
J	Compressione x z
C	Cambia directory prima di estrarre

Quindi: `-cvpf users.tar` = crea, verboso, **p**reserva permessi, **f**ile = `users.tar`
`-czf archivio.tar.gz` = crea, compressione **z** (gzip), **f**ile = `archivio.tar.gz`
`-xf archivio.tar.gz` = estrai, **f**ile = `archivio.tar.gz` (verbose omissa)

Il trattino (`-cvpf` vs `cvpf`) è opzionale per `tar` — entrambe le forme funzionano.

Path relativi — perché tar rimuove lo slash iniziale

Quando archivi `/home/alice`, `tar` potrebbe salvare il path assoluto `/home/alice/file.txt`. Al momento dell'estrazione, ricreerebbe esattamente quel path — sovrascrivendo il vero `/home/alice/file.txt` sul sistema di destinazione, anche se non è quello che vuoi.

Per sicurezza, `tar` rimuove lo slash iniziale e salva `home/alice/file.txt` (path relativo). Quando estrai in `/newdisk` con `-C /newdisk`, il file viene creato in `/newdisk/home/alice/file.txt` — senza toccare il sistema originale.

Archivio senza slash: `home/alice/file.txt`

^
path relativo → sicuro da estrarre ovunque

/newdisk non esiste — errore e soluzione

L'errore che hai visto (`No such file or directory`) è normale: `tar` non crea la directory di destinazione, deve esistere già. Soluzione:

```
sudo mkdir /newdisk
tar -C /newdisk -xvpf users.tar
```

Gli errori `Permission denied` su `.bash_history` sono normali: quei file appartengono ai singoli utenti e `vagrant` non può leggerli. Per archiviare anche quelli serve `sudo`:

```
sudo tar cvpf users.tar /home/*
```

La pipeline con -

```
tar cvpf - /home/* | tar -C /newdisk -xvpf -  
#           ^           ^  
#           - al posto del nome file      - al posto del nome file
```

In tar, -f - significa “usa stdin/stdout come archivio invece di un file su disco”.

- tar cvpf - /home/* → crea l’archivio e lo scrive su **stdout** (il - dopo f)
- | → collega lo stdout del primo comando allo stdin del secondo
- tar -C /newdisk -xvpf - → legge l’archivio da **stdin** (il - finale) ed estrae in /newdisk

Il risultato è identico a creare `users.tar` e poi estrarlo, ma senza creare il file intermedio — utile quando non hai spazio per tenere sia l’archivio che i file estratti.

Compressione con tar

Le opzioni z, j, J si aggiungono semplicemente alle combinazioni già viste:

```
tar czf archivio.tar.gz /home/*    # c + z(gzip) + f  
tar cjf archivio.tar.bz2 /home/*    # c + j(bzip2) + f  
tar cJf archivio.tar.xz /home/*     # c + J(xz) + f
```

Per estrarre, tar xf rileva automaticamente il formato di compressione — non devi specificare z/j/J:

```
tar xf archivio.tar.gz    # tar capisce da solo che è gzip  
tar xf archivio.tar.xz    # idem per xz
```

La pipeline con xz

```
tar cf - * | xz -c > archivio.tar.xz
```

- tar cf - * → crea archivio non compresso su stdout
- xz -c → comprime da stdin e scrive su stdout (-c = write to stdout)
- > archivio.tar.xz → dirige l’output compresso su file

L’output è silenzioso perché non c’è -v — sta lavorando, non è bloccato. Puoi aggiungere -v a tar se vuoi vedere i file processati.

rsync

rsync sincronizza directory trasferendo solo le differenze — non ricopia file già presenti a destinazione identici.

```
rsync -av /home/alice/ /backup/alice/      # locale
rsync -av /home/alice/ user@host:/backup/  # via SSH
```

-a (archive) è la combinazione più usata: ricorsivo + preserva permessi, timestamp, owner, group, link simbolici, file speciali. Quasi sempre si usa -av (aggiungendo verbose).

Esercizi 5 e 6 — script con find

mkdir -p — cosa fa il -p

Senza -p: mkdir /a/b/c fallisce se /a/b non esiste già, e fallisce anche se /a/b/c esiste già.

Con -p: - Crea tutte le directory intermedie mancanti (/a, poi /a/b, poi /a/b/c)
- Non dà errore se la directory esiste già

È quasi sempre la forma da usare negli script per evitare errori inutili.

Esercizio 5 — analisi riga per riga

```
find "$SRC" -maxdepth 1 -type f -mtime -"$GIORNI" -exec cp {} "$DST" \;
```

Smontata:

Parte	Significato
find "\$SRC"	Cerca dentro la directory sorgente
-maxdepth 1	Non scendere nelle sottodirectory — solo i file direttamente dentro \$SRC
-type f	Solo file regolari (escludi directory, link, ecc.)
-mtime -"\$GIORNI"	Modificati negli ultimi N giorni
-exec cp {} "\$DST" \;	Per ogni file trovato: esegui cp <file> \$DST

Il risultato è “copia flat”: tutti i file finiscono direttamente in \$DST, senza ricreare la struttura di sottodirectory dell’originale.

Esercizio 6 — cp --parents

```
find / -type f -mtime -"$GIORNI" -exec cp --parents {} "$DST" \;
```

cp --parents preserva il path completo nella destinazione:

Senza --parents:

```
cp /var/log/syslog /backup/ → /backup/syslog
```

Con --parents:

```
cp --parents /var/log/syslog /backup/ → /backup/var/log/syslog
```

Questo è ciò che si intende con “conservazione della struttura” — la gerarchia di directory originale viene ricreata dentro \$DST.